МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»
Московский приборостроительный техникум

ОТЧЕТ

по практическим работам №1, №2, №3, №4, №5

Тема: Программирование микроконтроллера Arduino Uno

Специальность 09.02.07 «Информационные системы и программирование»
код и наименование специальности

Студент: **Девлет Максим Керимович**
(фамилия, имя, отчество)

Группа: П-1-24

Руководитель по дисциплине:
Белова Дарья Романовна
(фамилия, имя, отчество)

«09» ноября 2026 года

СОДЕРЖАНИЕ

Практическая работа №1. Моделирование работы светофора на базе Arduino3

1.1. Цель работы ... 3
1.2. Схема подключения в эмуляторе ... 3
1.3. Исходный код программы ... 3
1.4. Краткое описание алгоритма ... 4
1.5. Вывод ... 4

Практическая работа №2. Управление яркостью и режимами светодиода с помощью кнопок и Serial-порта ... 5

2.1. Цель работы ... 5
2.2. Ссылка на интерактивный проект в Tinkercad ... 5
2.3. Схема подключения в эмуляторе ... 5
2.4. Исходный код программы ... 6
2.5. Краткое описание алгоритма ... 7
2.6. Вывод ... 8

Практическая работа №3. Работа с энергонезависимой памятью EEPROM 9

Практическая работа №4. Аппаратные прерывания и измерение скорости реакции 13

Практическая работа №5. Аналоговые датчики, analogRead(), map() 17

ПРАКТИЧЕСКАЯ РАБОТА №1

Тема: Моделирование работы светофора на базе микроконтроллера Arduino

1.1. Цель работы: изучить принципы управления цифровыми выходами микроконтроллера Arduino, собрать схему светофора в эмуляторе и запрограммировать базовый алгоритм его работы (переключение красного, желтого и зеленого сигналов с фазой мигания зеленого).

1.2. Схема подключения в эмуляторе

Схема светофора собрана на базе платы Arduino Uno. Светодиоды подключены к цифровым пинам через токоограничивающие резисторы:

• Красный светодиод (Red) — пин 8;
• Желтый светодиод (Yellow) — пин 9;
• Зеленый светодиод (Green) — пин 10.

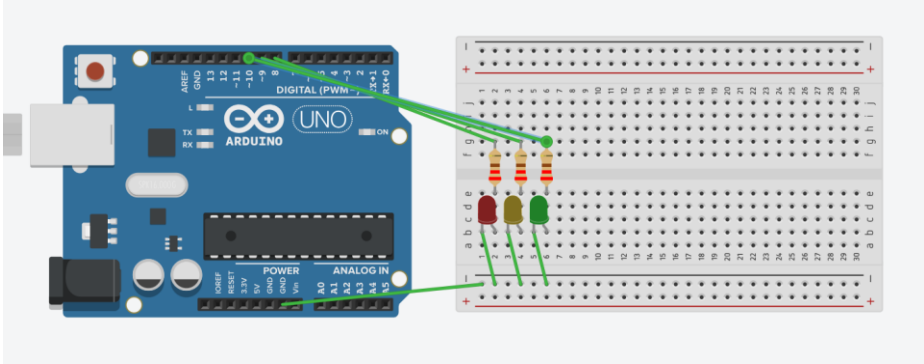


Рис. 1. Схема подключения светофора в эмуляторе

1.3. Исходный код программы

```
const int RED_PIN    = 8;
const int YELLOW_PIN = 9;
const int GREEN_PIN  = 10;

void setup() {
  pinMode(RED_PIN, OUTPUT);
  pinMode(YELLOW_PIN, OUTPUT);
  pinMode(GREEN_PIN, OUTPUT);
}

void loop() {
  // 1. Красный сигнал (3 сек)
  digitalWrite(RED_PIN, HIGH);
  digitalWrite(YELLOW_PIN, LOW);
  digitalWrite(GREEN_PIN, LOW);
  delay(3000);
```

```
  // 2. Желтый сигнал (1.5 сек)
  digitalWrite(RED_PIN, LOW);
  digitalWrite(YELLOW_PIN, HIGH);
  digitalWrite(GREEN_PIN, LOW);
  delay(1500);

  // 3. Зеленый сигнал (3 сек)
  digitalWrite(RED_PIN, LOW);
  digitalWrite(YELLOW_PIN, LOW);
  digitalWrite(GREEN_PIN, HIGH);
  delay(3000);

  // 4. Мигающий зеленый сигнал (3 раза)
  for (int i = 0; i < 3; i++) {
    digitalWrite(GREEN_PIN, LOW);
    delay(400);
    digitalWrite(GREEN_PIN, HIGH);
    delay(400);
  }
  digitalWrite(GREEN_PIN, LOW);

  // 5. Желтый сигнал перед переключением на красный (1.5 сек)
  digitalWrite(YELLOW_PIN, HIGH);
  delay(1500);
  digitalWrite(YELLOW_PIN, LOW);
}
```

1.4. Краткое описание алгоритма

В функции loop() реализован циклический алгоритм переключения фаз:
1. Горит красный (3 с) — запрещающий сигнал.
2. Включается желтый (1.5 с) — подготовка к движению.
3. Загорается зеленый (3 с) — разрешающий сигнал.
4. Зеленый мигает 3 раза (период 400 мс) — предупреждение о скором завершении зеленого сигнала.
5. Включается желтый (1.5 с) — предупреждающий сигнал перед повторным включением красного.

1.5. Вывод: в ходе выполнения лабораторной работы была успешно смоделирована и протестирована схема управления светофором на базе платформы Arduino. Закреплены практические навыки настройки цифровых пинов ввода-вывода (pinMode), управления дискретными сигналами (digitalWrite) и временными интервалами (delay).

ПРАКТИЧЕСКАЯ РАБОТА №2

Тема: Управление яркостью и режимами работы светодиода с помощью кнопок и Serial-порта

2.1. Цель работы: изучить принципы подключения тактовых кнопок с использованием внутренней подтяжки (INPUT_PULLUP), регулирования яркости светодиода с помощью широтно-импульсной модуляции (ШИМ / PWM, analogWrite), неблокирующей задержки на основе таймера millis(), а также приёма и передачи данных через последовательный интерфейс (Serial-порт) на платформе Arduino.

2.2. Ссылка на интерактивный проект в Tinkercad

Интерактивный проект схемы и программного кода доступен в эмуляторе Tinkercad по ссылке:
https://www.tinkercad.com/things/lSEULdbWDn3-brilliant-blorr-gogo?sharecode=06HgGk8-THOgBvE3u8T8ayUC4vQrngoF99phC12BjA0

2.3. Схема подключения в эмуляторе

Схема собрана на базе микроконтроллера Arduino Uno в эмуляторе Tinkercad. Подключение компонентов выполнено следующим образом:
• Светодиод (LED) подключен катодом к GND, а анодом через токоограничивающий резистор (220 Ом) — к цифровому ШИМ-выводу 9 (LED_PIN);
• Кнопка 1 (Btn1, включение / выключение питания) подключена к цифровому пину 2 (BTN1_PIN) и к выводу GND (режим INPUT_PULLUP);
• Кнопка 2 (Btn2, ступенчатая регулировка яркости ШИМ) подключена к цифровому пину 3 (BTN2_PIN) и к выводу GND (режим INPUT_PULLUP);
• Кнопка 3 (Btn3, вывод имени студента в Serial) подключена к цифровому пину 4 (BTN3_PIN) и к выводу GND (режим INPUT_PULLUP);
• Связь по виртуальному Serial-порту осуществляется на скорости 9600 бод.

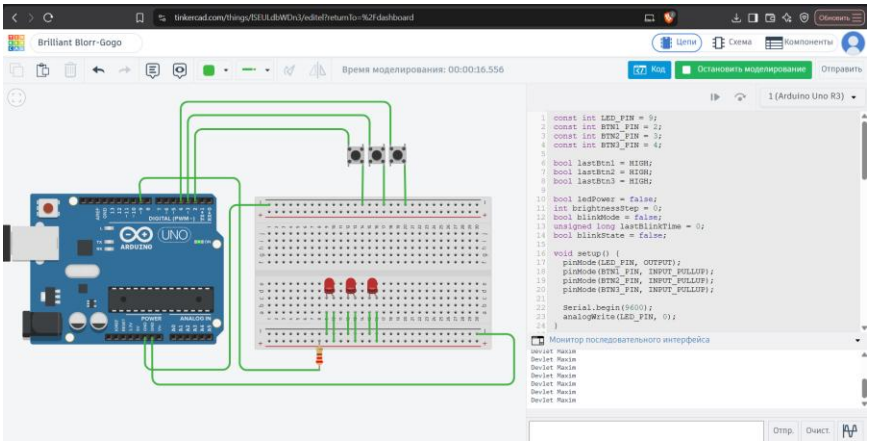


Рис. 2. Схема подключения в эмуляторе Tinkercad (Практическая работа №2)

2.4. Исходный код программы

```
const int LED_PIN = 9;
const int BTN1_PIN = 2;
const int BTN2_PIN = 3;
const int BTN3_PIN = 4;

bool lastBtn1 = HIGH;
bool lastBtn2 = HIGH;
bool lastBtn3 = HIGH;

bool ledPower = false;
int brightnessStep = 0;
bool blinkMode = false;
unsigned long lastBlinkTime = 0;
bool blinkState = false;

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BTN1_PIN, INPUT_PULLUP);
  pinMode(BTN2_PIN, INPUT_PULLUP);
  pinMode(BTN3_PIN, INPUT_PULLUP);

  Serial.begin(9600);
  analogWrite(LED_PIN, 0);
}

void loop() {
  if (Serial.available() > 0) {
    char cmd = Serial.read();
    if (cmd == 'M' || cmd == 'm') {
      blinkMode = true;
    } else if (cmd == 'S' || cmd == 's') {
      blinkMode = false;
      ledPower = false;
      brightnessStep = 0;
      analogWrite(LED_PIN, 0);
    }
  }
```

```
  bool curBtn1 = digitalRead(BTN1_PIN);
  if (lastBtn1 == HIGH && curBtn1 == LOW) {
    blinkMode = false;
    ledPower = !ledPower;
    if (ledPower) {
      brightnessStep = 5;
      analogWrite(LED_PIN, 255);
    } else {
      brightnessStep = 0;
      analogWrite(LED_PIN, 0);
    }
    delay(50);
  }
  lastBtn1 = curBtn1;

  bool curBtn2 = digitalRead(BTN2_PIN);
  if (lastBtn2 == HIGH && curBtn2 == LOW) {
    blinkMode = false;
    brightnessStep++;
    if (brightnessStep > 5) {
      brightnessStep = 0;
      ledPower = false;
    } else {
      ledPower = true;
    }
    int pwmVal = brightnessStep * 51;
    analogWrite(LED_PIN, pwmVal);
    delay(50);
  }
  lastBtn2 = curBtn2;

  bool curBtn3 = digitalRead(BTN3_PIN);
  if (lastBtn3 == HIGH && curBtn3 == LOW) {
    Serial.println("Devlet Maxim");
    delay(50);
  }
  lastBtn3 = curBtn3;

  if (blinkMode) {
    if (millis() - lastBlinkTime >= 300) {
      lastBlinkTime = millis();
      blinkState = !blinkState;
      analogWrite(LED_PIN, blinkState ? 255 : 0);
    }
  }
}
```

2.5. Краткое описание алгоритма

В функциях setup() и loop() реализована комплексная логика управления:
1. Инициализация (setup): цифровой пин 9 (светодиод) переводится в режим вывода (OUTPUT), пины 2, 3 и 4 (кнопки) настраиваются на вход с внутренней подтяжкой (INPUT_PULLUP), что исключает необходимость во внешних резисторах. Открывается последовательный порт на скорости 9600 бод

(Serial.begin(9600)), начальное состояние светодиода — выключенное (analogWrite(LED_PIN, 0)).
2. Управление через Serial-порт: при поступлении данных в буфер (Serial.available() > 0) считывается управляющий символ (Serial.read()). Символ 'M' или 'm' активирует режим мигания (blinkMode = true). Символ 'S' или 's' останавливает мигание, выключает светодиод и сбрасывает все переменные состояния (blinkMode = false, ledPower = false, brightnessStep = 0, analogWrite(LED_PIN, 0)).
3. Кнопка 1 (пин 2 — питание): регистрируется нажатие (переход HIGH -> LOW) с защитой от дребезга контактов (delay 50 мс). Кнопка сбрасывает мигание и переключает состояние питания светодиода (ledPower): при включении устанавливает максимальную яркость (шаг 5, ШИМ 255), при выключении гасит светодиод.
4. Кнопка 2 (пин 3 — регулировка яркости): регистрируется нажатие с защитой от дребезга (delay 50 мс). Сбрасывает мигание и ступенчато увеличивает яркость (brightnessStep++). Уровень ШИМ рассчитывается как brightnessStep * 51 (формируя 6 уровней: 0, 51, 102, 153, 204, 255). При превышении значения 5 яркость сбрасывается в 0 (выключение).
5. Кнопка 3 (пин 4 — вывод имени): при нажатии передаёт через монитор последовательного порта строку "Devlet Maxim" (Serial.println).
6. Неблокирующий режим мигания: при активном флаге blinkMode с помощью системного таймера millis() каждые 300 мс инвертируется логическое состояние blinkState (светодиод циклически зажигается на полную яркость и гаснет), не препятствуя приёму команд по Serial и опросу кнопок.

2.6. Вывод: в ходе выполнения практической работы были закреплены навыки программирования микроконтроллера Arduino и работы с его аппаратными интерфейсами. Изучено подключение тактовых кнопок по схеме с внутренней подтяжкой (INPUT_PULLUP) и программным устранением дребезга контактов, освоено управление мощностью и яркостью светодиода методом широтно-импульсной модуляции (ШИМ / PWM, функция analogWrite), построение неблокирующих временных задержек на базе функции millis(), а также двусторонний обмен данными с персональным компьютером через последовательный порт Serial для вывода текстовых сообщений и приёма команд управления. Все функциональные требования успешно реализованы и проверены в среде моделирования Tinkercad.

ПРАКТИЧЕСКАЯ РАБОТА №3

Тема: Работа с энергонезависимой памятью EEPROM

3.1. Цель работы: изучить назначение, архитектуру и способы взаимодействия с энергонезависимой памятью EEPROM микроконтроллера ATmega328P на платформе Arduino. Освоить безопасные алгоритмы записи, чтения и обнуления данных с использованием библиотеки <EEPROM.h>, построить схему подсчета нажатий кнопок с программным антидребезгом, организовать четную/нечетную индикацию на 4 светодиодах и дублирование состояния в последовательный порт.

3.2. Ссылка на интерактивный проект в Tinkercad

Интерактивный проект схемы и программного кода доступен в эмуляторе Tinkercad по ссылке:
https://app.cirkitdesigner.com/project/ec7bb51e-8433-4b15-ada7-2f1b249e6964

3.3. Схема подключения в эмуляторе

Схема собрана на базе микроконтроллера Arduino Uno в среде эмуляции Tinkercad / Wokwi. Подключение внешних компонентов организовано следующим образом: Светодиод D1 (зеленый / четное S1): анод через резистор 220 Ом к цифровому пину 8 (D1_PIN), катод на GND; Светодиод D2 (красный / нечетное S1): анод через резистор 220 Ом к цифровому пину 9 (D2_PIN), катод на GND; Светодиод D3 (зеленый / четное S2): анод через резистор 220 Ом к цифровому пину 10 (D3_PIN), катод на GND; Светодиод D4 (красный / нечетное S2): анод через резистор 220 Ом к цифровому пину 11 (D4_PIN), катод на GND; Кнопка S1 (инкремент счетчика S1): подключена к цифровому пину 2 (S1_PIN) и к GND (режим INPUT_PULLUP); Кнопка S2 (инкремент счетчика S2): подключена к цифровому пину 3 (S2_PIN) и к GND (режим INPUT_PULLUP); Кнопка S3 (обнуление ячеек EEPROM): подключена к цифровому пину 4 (S3_PIN) и к GND (режим INPUT_PULLUP); Скорость передачи телеметрии в COM-порт (UART): 9600 бод.

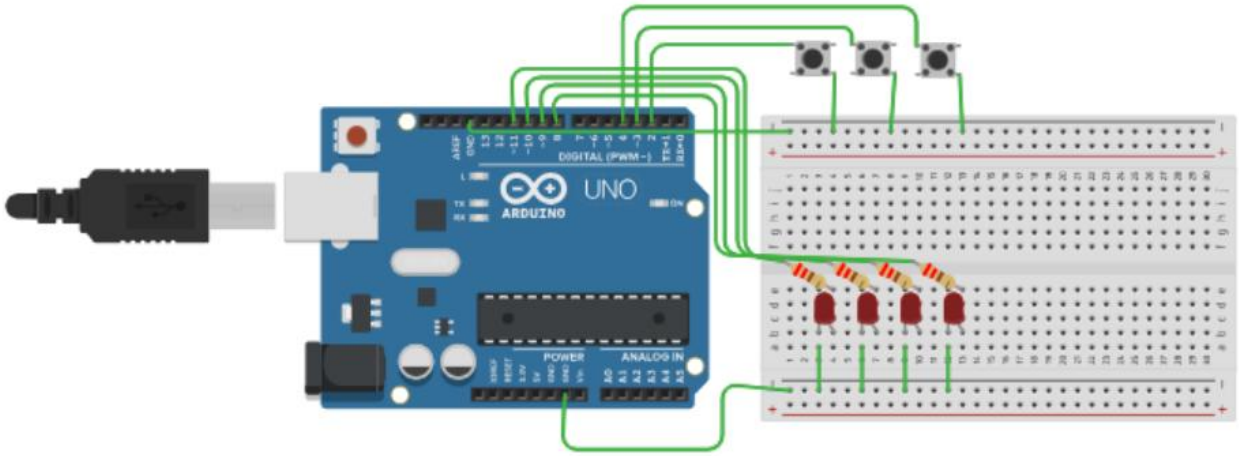


Рис. 3. Схема подключения кнопок и светодиодов к Arduino Uno (Практическая работа №3)

3.3. Исходный код программы

```
#include <EEPROM.h>

const int PIN_S1 = 2;
const int PIN_S2 = 3;
const int PIN_S3 = 4;

const int PIN_D1 = 8;
const int PIN_D2 = 9;
const int PIN_D3 = 10;
const int PIN_D4 = 11;

const int ADDR_S1 = 0;
const int ADDR_S2 = 2;
int countS1 = 0;
int countS2 = 0;

bool lastS1 = HIGH;
bool lastS2 = HIGH;
bool lastS3 = HIGH;

void updateLeds() {
  if (countS1 % 2 == 0) {
    digitalWrite(PIN_D1, HIGH);
    digitalWrite(PIN_D2, LOW);
```

```
  } else {
    digitalWrite(PIN_D1, LOW);
    digitalWrite(PIN_D2, HIGH);
  }

  if (countS2 % 2 == 0) {
    digitalWrite(PIN_D3, HIGH);
    digitalWrite(PIN_D4, LOW);
  } else {
    digitalWrite(PIN_D3, LOW);
    digitalWrite(PIN_D4, HIGH);
  }
}

void printStatus() {
  Serial.print("S1 count: ");
  Serial.print(countS1);
  Serial.print(" | S2 count: ");
  Serial.println(countS2);
}

void setup() {
  Serial.begin(9600);

  pinMode(PIN_S1, INPUT_PULLUP);
  pinMode(PIN_S2, INPUT_PULLUP);
  pinMode(PIN_S3, INPUT_PULLUP);

  pinMode(PIN_D1, OUTPUT);
  pinMode(PIN_D2, OUTPUT);
  pinMode(PIN_D3, OUTPUT);
  pinMode(PIN_D4, OUTPUT);

  EEPROM.get(ADDR_S1, countS1);
  EEPROM.get(ADDR_S2, countS2);

  if (countS1 < 0) countS1 = 0;
  if (countS2 < 0) countS2 = 0;

  updateLeds();
  printStatus();
}

void loop() {
```

```
  bool curS1 = digitalRead(PIN_S1);
  if (lastS1 == HIGH && curS1 == LOW) {
    countS1++;
    EEPROM.put(ADDR_S1, countS1);
    updateLeds();
    printStatus();
    delay(50);
  }
  lastS1 = curS1;

  bool curS2 = digitalRead(PIN_S2);
  if (lastS2 == HIGH && curS2 == LOW) {
    countS2++;
    EEPROM.put(ADDR_S2, countS2);
    updateLeds();
    printStatus();
    delay(50);
  }
  lastS2 = curS2;

  bool curS3 = digitalRead(PIN_S3);
  if (lastS3 == HIGH && curS3 == LOW) {
    countS1 = 0;
    countS2 = 0;
    EEPROM.put(ADDR_S1, countS1);
    EEPROM.put(ADDR_S2, countS2);
    Serial.println("--- EEPROM RESET ---");
    updateLeds();
    printStatus();
    delay(50);
  }
  lastS3 = curS3;
}
```

3.4. Краткое описание алгоритма

В функциях setup() и loop() реализован отказоустойчивый алгоритм взаимодействия с периферией и памятью:
1. Инициализация (setup): цифровые линии светодиодов 8–11 переводятся в режим вывода (OUTPUT), а выводы кнопок 2–4 конфигурируются со встроенной подтяжкой (INPUT_PULLUP). Инициализируется UART-порт на скорости 9600 бод. По адресу 0x00 считывается сигнатура: если она не равна 0xAA (первый пуск чипа), ячейки инициализируются нулями и записывается маркер; иначе

сохраненные значения счетчиков загружаются из EEPROM с помощью шаблона EEPROM.get().
2. Обработка кнопок S1 и S2: фиксируется спадающий фронт сигнала (переход HIGH -> LOW) с защитной задержкой от дребезга 50 мс. Счетчик инкрементируется и перезаписывается в память функцией EEPROM.put(), использующей метод update (перезапись только изменившихся байтов для сбережения ресурса ячеек памяти).
3. Логика индикации четности (updateLeds): вычисляется остаток от деления счетчика на 2. Для S1 при четном значении (включая 0) включается D1, а D2 гаснет; при нечетном — включается D2, а D1 гаснет. Для S2 аналогично: при четном горит D3, гаснет D4; при нечетном горит D4, гаснет D3.
4. Сброс кнопкой S3: при нажатии клавиши S3 счетчики countS1 и countS2 обнуляются в ОЗУ, нули записываются в соответствующие ячейки EEPROM, индикация возвращается в базовый четный режим, а в последовательный порт транслируется сообщение об успешной очистке.

3.5. Вывод: в ходе выполнения практической работы были закреплены навыки программирования микроконтроллера ATmega328P и работы со встроенной энергонезависимой памятью EEPROM. Освоено использование библиотеки <EEPROM.h> для сохранения и загрузки 16-битных данных (EEPROM.put / EEPROM.get) с сохранением физического ресурса ячеек. Успешно реализована схема из трех кнопок с подтяжкой INPUT_PULLUP и четырех светодиодов, сформирован алгоритм раздельной индикации четности нажатий клавиш S1 и S2, работающий непрерывно после перезагрузки питания, а также сервисная функция очистки памяти по кнопке S3 с дублированием телеметрии в COM-порт.

ПРАКТИЧЕСКАЯ РАБОТА №4

Тема: Аппаратные прерывания и измерение скорости реакции

4.1. Цель работы: изучить назначение, архитектуру и аппаратные принципы функционирования внешних прерываний (External Interrupts, функции attachInterrupt(), digitalPinToInterrupt()) в микроконтроллерах ATmega328P на платформе Arduino. Освоить безопасные алгоритмы обработки асинхронных сигналов в процедуре ISR с использованием спецификатора volatile и программного подавления дребезга контактов (debounce). Спроектировать и смоделировать интерактивное устройство «Тренажер скорости реакции» с неблокирующим измерением интервалов времени (millis()), псевдослучайной задержкой ожидания визуального стимула (random(), randomSeed(analogRead(A0))), серией из 5 раундов, автоматическим определением рекорда, расчетом среднего времени отклика и выводом подробной статистики в последовательный порт UART.

4.2. Схема подключения в эмуляторе

Схема собрана на базе микроконтроллера Arduino Uno в среде эмуляции Tinkercad / Wokwi. Подключение внешних компонентов организовано следующим образом:
• Светодиод (LED): анод через токоограничивающий резистор 220 Ом подключен к цифровому выводу 8 (ledPin), катод соединен с шиной GND;
• Тактовая кнопка: подключена к цифровому выводу 2 (buttonPin) и к шине GND в режиме со встроенной подтяжкой INPUT_PULLUP (аппаратное прерывание INT0, фиксация нажатия по спадающему фронту FALLING);
• Аналоговый вход A0: оставлен неподключенным (плавающий вход) для считывания атмосферных шумов и первичной инициализации генератора случайных чисел randomSeed();
• Скорость передачи телеметрии в COM-порт (UART): 9600 бод.

Таблица 2. Распределение выводов и параметров схемы тренажера скорости реакции

Пин Arduino	Подключенный компонент	Назначение и логика работы	Режим (Mode)
D8	Светодиод (LED) через резистор 220 Ом	Визуальный стимул: включается через случайный интервал времени (2–5 с)	OUTPUT
D2 (INT0)	Тактовая кнопка (замыкание на GND)	Фиксация реакции игрока; аппаратное прерывание по спадающему фронту (FALLING)	INPUT_PULLUP
A0	Плавающий контакт	Считывание фонового аналогового	INPUT (Analog)

		шума для генерации случайных чисел (randomSeed)	
GND	Общая шина питания	Замыкание катода светодиода и контакта тактовой кнопки	Power
UART	Serial Monitor	Вывод телеметрии раундов, времени реакции, рекорда и итоговой статистики	9600 бод

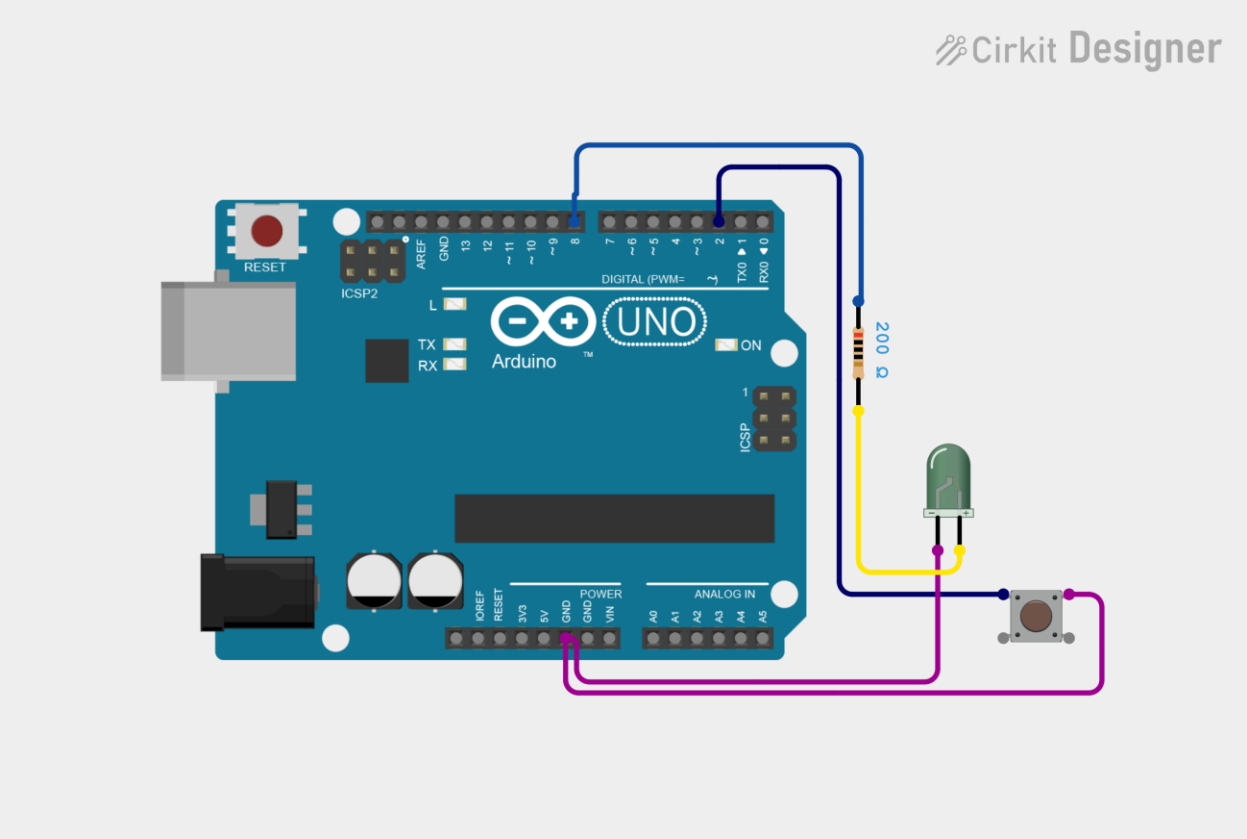


Рис. 4. Схема подключения тренажера скорости реакции к Arduino Uno (Практическая работа №4)

4.3. Исходный код программы

```
const int ledPin = 8;
const int buttonPin = 2;

volatile bool buttonPressed = false;
unsigned long lastPressTime = 0;

unsigned long waitStartMillis = 0;
unsigned long waitInterval = 0;
unsigned long ledTurnOnMillis = 0;

bool isWaitingForLed = true;
int currentRound = 0;
unsigned long results[5];
unsigned long bestTime = 999999;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  Serial.begin(9600);

  attachInterrupt(digitalPinToInterrupt(buttonPin), onButton, FALLING);

  randomSeed(analogRead(A0));

  Serial.println("Game Started");
  Serial.println("Wait until led on and press button");

  startNewRound();
}

void loop() {
  unsigned long now = millis();

  if (isWaitingForLed) {
    if (buttonPressed) {
      buttonPressed = false;
    }

    if (now - waitStartMillis >= waitInterval) {
      digitalWrite(ledPin, HIGH);
      ledTurnOnMillis = now;
      isWaitingForLed = false;
      buttonPressed = false;
    }
  }
  else {
    if (buttonPressed) {
      buttonPressed = false;

      unsigned long reactionTime = now - ledTurnOnMillis;
      digitalWrite(ledPin, LOW);

      results[currentRound] = reactionTime;
      currentRound++;

      Serial.print("Round ");
      Serial.print(currentRound);
      Serial.print(": ");
```

```
      Serial.print(reactionTime);
      Serial.println(" ms");

      if (reactionTime < bestTime) {
        bestTime = reactionTime;
        Serial.println("New Record!");
      }

      if (currentRound == 5) {
        printFinalStats();
        currentRound = 0;
        Serial.println("\nNEW GAME");
      }

      startNewRound();
    }
  }
}

void startNewRound() {
  waitStartMillis = millis();
  waitInterval = random(2000, 5001);
  isWaitingForLed = true;
}

void printFinalStats() {
  Serial.println("Results");
  unsigned long sum = 0;

  Serial.print("All Results ");
  for (int i = 0; i < 5; i++) {
    Serial.print(results[i]);
    Serial.print(" ms");
    if (i < 4) Serial.print(", ");
    sum += results[i];
  }
  Serial.println();

  Serial.print("Best Result: ");
  Serial.print(bestTime);
  Serial.println(" ms");

  unsigned long averageTime = sum / 5;
  Serial.print("Avg Reaction time ");
  Serial.print(averageTime);
  Serial.println(" ms");
  Serial.println("----------------------");
}

void onButton() {
  unsigned long now = millis();
  if (now - lastPressTime > 200) {
    buttonPressed = true;
    lastPressTime = now;
  }
}
```

4.4. Краткое описание алгоритма

В функциях setup(), loop() и процедуре прерывания onButton() реализован надежный неблокирующий алгоритм управления игровым процессом:
1. Инициализация и регистрация прерывания (setup): цифровой пин 8 переводится в режим вывода (OUTPUT), пин 2 настраивается со встроенной подтяжкой (INPUT_PULLUP). Функция attachInterrupt(digitalPinToInterrupt(buttonPin), onButton, FALLING) связывает аппаратное прерывание INT0 с функцией onButton() по перепаду HIGH -> LOW. Неподключенный вход A0 используется для инициализации randomSeed(analogRead(A0)), что исключает повторение задержек при перезапуске.
2. Обработчик аппаратного прерывания (onButton): срабатывает асинхронно при замыкании кнопки. Проверка now - lastPressTime > 200 отсекает высокочастотный дребезг механических контактов кнопки. Флаг buttonPressed объявлен как volatile, что гарантирует прямое считывание из ОЗУ без регистровой оптимизации компилятором.
3. Фаза ожидания и защита от фальстарта (isWaitingForLed == true): интервал ожидания задается случайно от 2 до 5 с (random(2000, 5001)). Если кнопка нажата до включения светодиода, флаг сбрасывается без регистрации попытки. По истечении таймера зажигается светодиод, фиксируется метка времени ledTurnOnMillis = now, и система переходит в режим замера реакции.
4. Замер времени реакции и учет рекорда (isWaitingForLed == false): при нажатии рассчитывается интервал отклика reactionTime = now - ledTurnOnMillis, светодиод гаснет, результат заносится в массив results[currentRound], обновляется счетчик раундов. При побитии рекорда (reactionTime < bestTime) фиксируется новый рекорд и отправляется уведомление в Serial-порт.
5. Статистический анализ серии (printFinalStats): по достижении 5 раундов вычисляется среднее время реакции averageTime = sum / 5, в консоль выводятся результаты всех попыток, лучшее и среднее время, после чего счетчик раундов обнуляется и стартует новая игра.

4.5. Вывод: в ходе выполнения практической работы были всесторонне освоены механизмы обработки внешних аппаратных прерываний в микроконтроллерах ATmega328P на платформе Arduino. Собрана принципиальная схема тренажера скорости реакции с тактовой кнопкой на выводе внешнего прерывания INT0 (пин 2) и светодиодом индикации (пин 8). Успешно применена функция attachInterrupt() с триггером по спадающему фронту (FALLING), обеспечена безопасность межпоточного флага с помощью спецификатора volatile и реализован фильтр подавления дребезга контактов. Разработан неблокирующий

алгоритм на базе millis() со случайным интервалом ожидания стимула (2–5 с), защитой от фальстартов, автофиксацией рекорда и статистической обработкой серии из 5 раундов с выводом телеметрии в COM-порт.

ПРАКТИЧЕСКАЯ РАБОТА №5

Тема: Аналоговые датчики, analogRead(), map()

5.1. Цель работы: изучить принципы функционирования аналоговых датчиков и основы аналого-цифрового преобразования (АЦП) на базе 10-разрядного АЦП последовательного приближения (SAR ADC) микроконтроллера ATmega328P платформы Arduino. Освоить практическое применение встроенных функций считывания аналогового напряжения analogRead(), ограничения диапазона constrain() и линейного масштабирования map(). Спроектировать и смоделировать аппаратно-программную систему автомобильного парктроника («датчик парковки») на основе фоторезистора, пьезоизлучателя, светодиода и тактовой кнопки с неблокирующей логикой работы (millis()), автоматической 3-секундной калибровкой базового уровня освещенности, зависимостью частоты импульсов от приближения руки, режимом непрерывной аварийной сигнализации («СТОП!») при сближении менее 10% и асинхронным отключением звукового сопровождения через аппаратное прерывание (attachInterrupt()).

5.2. Схема подключения в эмуляторе

Интерактивный проект схемы доступен в среде Cirkit Designer по ссылке: https://app.cirkitdesigner.com/project/41090fec-b92a-40f8-b21c-a15b956f2572

Схема устройства смоделирована на базе микроконтроллера Arduino Uno в среде симуляции Cirkit Designer / Tinkercad. Подключение внешних компонентов организовано следующим образом:
• Фоторезистор (LDR): включен по схеме резистивного делителя напряжения с постоянным резистором 10 кОм (вывод к +5V, средняя точка к аналоговому входу A0, резистор 10 кОм к GND). При приближении руки освещенность падает, сопротивление растет, напряжение на входе A0 снижается;
• Тактовая кнопка: подключена между цифровым выводом 2 (BUTTON_PIN) и шиной GND со встроенной подтяжкой INPUT_PULLUP (аппаратное прерывание INT0, фиксация нажатия по спадающему фронту FALLING);
• Пьезоизлучатель (Buzzer): подключен к цифровому выводу 8 (BUZZER_PIN) и шине GND. Генерация звукового сигнала частотой 2000 Гц осуществляется функцией tone(), выключение — noTone();
• Светодиод (LED): анод через токоограничивающий резистор 220 Ом подключен к цифровому выводу 9 (LED_PIN), катод соединен с шиной GND;
• Скорость передачи телеметрии в COM-порт (UART): 9600 бод.

Таблица 1. Распределение выводов и параметров схемы аналогового парктроника

Пин Arduino	Подключенный компонент	Назначение и логика работы	Режим (Mode)
A0	Фоторезистор (LDR) + резистор 10 кОм	Делитель напряжения; измерение освещенности (0–1023)	INPUT (Analog ADC)
D2 (INT0)	Тактовая кнопка (замыкание на GND)	Переключение звука (Mute); аппаратное прерывание по фронту FALLING	INPUT_PULLUP
D8	Пьезоизлучатель (Buzzer)	Звуковая сигнализация (импульсы 2000 Гц / непрерывный тон при сближении)	OUTPUT
D9	Светодиод (LED) через резистор 220 Ом	Световая визуализация импульсов приближения и непрерывная тревога	OUTPUT
GND / +5V	Общие шины питания	Питание делителя фоторезистора и замыкание цепей периферии	Power
UART	Serial Monitor	Вывод телеметрии (каждые 300 мс) и предупреждения «СТОП!»	9600 бод

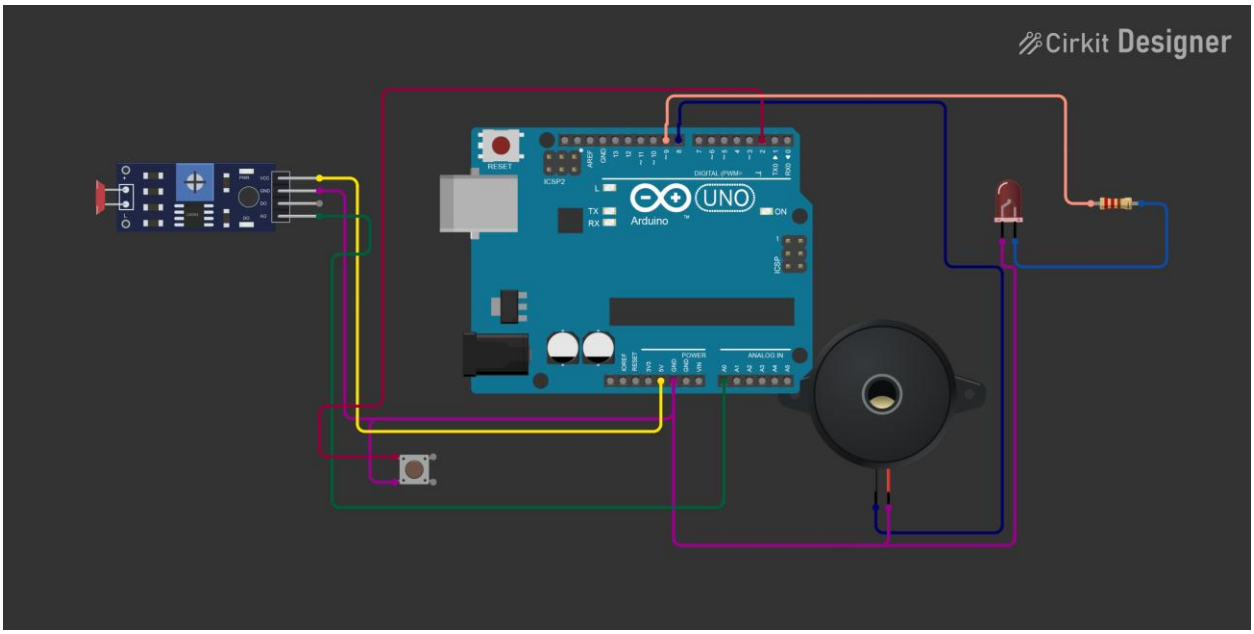


Рис. 1. Принципиальная схема подключения аналогового парктроника к Arduino Uno в эмуляторе

5.3. Исходный код программы

```
const uint8_t LDR_PIN = A0;
const uint8_t BUTTON_PIN = 2;
const uint8_t BUZZER_PIN = 8;
const uint8_t LED_PIN = 9;

const uint32_t CALIBRATION_MS = 3000UL;
const uint32_t DEBOUNCE_MS = 180UL;
const uint32_t SERIAL_INTERVAL_MS = 300UL;
const uint32_t BEEP_DURATION_MS = 60UL;
const uint16_t TONE_FREQ = 2000U;

uint32_t calibrateStartTime = 0;
uint32_t lastSignalTime = 0;
uint32_t lastSerialTime = 0;

volatile uint32_t lastButtonTime = 0;
volatile bool soundEnabled = true;

int calibratedLight = 0;
int minDark = 0;
uint32_t calibrationSum = 0;
uint16_t calibrationSamples = 0;

bool isCalibrated = false;
bool signalActive = false;

void toggleSound() {
  const uint32_t now = millis();
  if (now - lastButtonTime >= DEBOUNCE_MS) {
    soundEnabled = !soundEnabled;
    lastButtonTime = now;
  }
}

void setup() {
  pinMode(LDR_PIN, INPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);

  digitalWrite(LED_PIN, LOW);
  noTone(BUZZER_PIN);

  Serial.begin(9600);
  calibrateStartTime = millis();
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), toggleSound, FALLING);

  Serial.println(F("Калибровка освещения: 3 секунды..."));
}

void loop() {
  const uint32_t now = millis();

  if (!isCalibrated) {
    if (calibrationSamples < 60000U) {
      calibrationSum += analogRead(LDR_PIN);
      ++calibrationSamples;
    }
```

```
  if (now - calibrateStartTime >= CALIBRATION_MS) {
    if (calibrationSamples == 0) return;

    calibratedLight = calibrationSum / calibrationSamples;
    minDark = calibratedLight - 300;
    if (minDark < 0) minDark = 0;
    if (calibratedLight <= minDark) calibratedLight = minDark + 1;

    isCalibrated = true;
    Serial.print(F("Калибровка завершена; базовый ADC = "));
    Serial.println(calibratedLight);
  }
  return;
}

const int val = analogRead(LDR_PIN);
const int constrainedVal = constrain(val, minDark, calibratedLight);
const int distance = map(constrainedVal, minDark, calibratedLight, 0, 100);
const bool alarm = (distance < 10);

if (now - lastSerialTime >= SERIAL_INTERVAL_MS) {
  lastSerialTime = now;
  if (alarm) {
    Serial.println(F("СТОП!"));
  } else {
    Serial.print(distance);
    Serial.println(F("%"));
  }
}

if (alarm) {
  digitalWrite(LED_PIN, HIGH);
  if (soundEnabled) {
    tone(BUZZER_PIN, TONE_FREQ);
  } else {
    noTone(BUZZER_PIN);
  }
  signalActive = true;
  lastSignalTime = now;
} else {
  const uint32_t pauseMs = (uint32_t)map(distance, 10, 100, 100, 1000);

  if (signalActive) {
    if (now - lastSignalTime >= BEEP_DURATION_MS) {
      signalActive = false;
      lastSignalTime = now;
      digitalWrite(LED_PIN, LOW);
      noTone(BUZZER_PIN);
    } else if (!soundEnabled) {
      noTone(BUZZER_PIN);
    }
  } else {
    if (now - lastSignalTime >= pauseMs) {
      signalActive = true;
      lastSignalTime = now;
      digitalWrite(LED_PIN, HIGH);
      if (soundEnabled) {
        tone(BUZZER_PIN, TONE_FREQ);
      }
```

```
      }
    }
  }
}
```

5.4. Краткое описание алгоритма

В функциях setup(), loop() и процедуре прерывания toggleSound() реализован неблокирующий масштабируемый алгоритм управления парковочной системой:
1. Инициализация и регистрация прерывания (setup): аналоговый вход A0 настраивается на ввод (INPUT), пин кнопки 2 конфигурируется со встроенной подтяжкой (INPUT_PULLUP), выводы зуммера (8) и светодиода (9) переводятся в режим вывода (OUTPUT). Функция attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), toggleSound, FALLING) связывает внешнее прерывание INT0 с обработчиком кнопки. Устанавливается скорость UART 9600 бод и фиксируется метка времени старта калибровки.
2. Адаптивная автокалибровка (isCalibrated == false): в течение первых 3000 мс программа накапливает сумму отсчетов АЦП (calibrationSum) и число выборок (до 60000). По истечении таймера вычисляется среднее значение calibratedLight (базовый уровень освещенности помещения в состоянии «свободно»), а также адаптивный порог затемнения minDark = calibratedLight - 300 с защитой от выхода в отрицательную область.
3. Опрос датчика и расчет расстояния (loop): функция analogRead(LDR_PIN) считывает уровень сигнала. Через constrain(val, minDark, calibratedLight) исключаются случайные выбросы за пределы калибровки, а с помощью map(constrainedVal, minDark, calibratedLight, 0, 100) значение преобразуется в процентное «расстояние» от 0% («рука вплотную») до 100% («препятствия нет»). Вычисляется условие аварийной зоны alarm = (distance < 10).
4. Интервальная телеметрия UART: каждые 300 мс по системному таймеру (now - lastSerialTime >= SERIAL_INTERVAL_MS) в COM-порт выводится либо текущее расстояние в процентах, либо предупреждение «СТОП!», если активирован режим alarm.
5. Неблокирующая светозвуковая сигнализация: при distance < 10% светодиод горит постоянно, а зуммер выдает непрерывный тон 2000 Гц (если звук разрешен). При distance >= 10% длительность паузы между короткими импульсами (BEEP_DURATION_MS = 60 мс) динамически масштабируется через map(distance, 10, 100, 100, 1000) от 100 мс (близко) до 1000 мс (далеко). Переключение состояний выполняется через millis() без задержек delay().
6. Асинхронное переключение звука (toggleSound): по спадающему фронту

нажатия кнопки вызывается обработчик прерывания. Проверка now - lastButtonTime >= DEBOUNCE_MS (180 мс) подавляет дребезг контактов. Переменная soundEnabled инвертируется; спецификатор volatile гарантирует моментальное считывание актуального состояния в основном цикле, при этом светодиодная индикация продолжает функционировать независимо от звука.

5.5. Вывод: в ходе выполнения практической работы были всесторонне изучены принципы взаимодействия с аналоговыми датчиками и механизмы АЦП в микроконтроллерах ATmega328P на платформе Arduino. Спроектирована и промоделирована в среде Cirkit Designer схема автомобильного парктроника с фоторезистивным делителем на входе A0, кнопкой внешнего прерывания INT0 (пин 2), пьезоизлучателем (пин 8) и светодиодом (пин 9). Успешно применены функции analogRead(), constrain() и map() для пересчета аналогового сигнала в условную дистанцию от 0% до 100% и динамической регулировки интервалов звуковых импульсов от 100 до 1000 мс. Разработан полностью неблокирующий алгоритм на базе системного таймера millis(), включающий 3-секундную адаптивную автокалибровку под фоновый свет, непрерывную аварийную сигнализацию («СТОП!») при сближении менее 10%, периодический вывод телеметрии в последовательный COM-порт каждые 300 мс и асинхронное отключение звукового сопровождения через аппаратное прерывание с программным подавлением дребезга контактов.